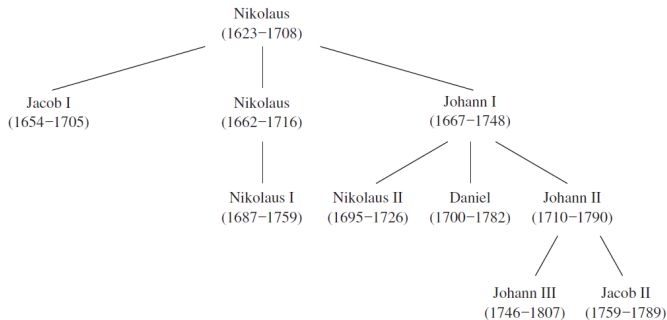


Introduction to trees

(Paragraph 11.1)

The undirected graph representing a family tree is an example of a tree.



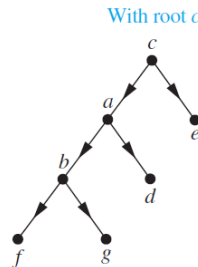
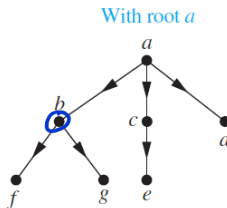
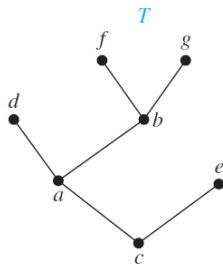
A tree cannot have:

- a simple circuit
- multiple edges
- loops
- isolated points

A **tree** is a connected undirected graph with no simple circuits.

Rooted trees

A **rooted tree** is a tree in which one vertex has been designated as the root and every edge is directed away from the root.

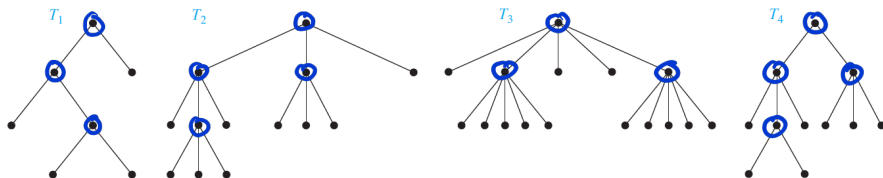


The **parent** of v is the unique vertex u s. t. there is a directed edge from u to v . When u is the parent of v , v is called a **child** of u . Vertices with the same parent are called **siblings**. The **ancestors** are the vertices in the path from the root to vertex, excluding the vertex itself and including the root. The **descendants** of a vertex v are those vertices that have v as an ancestor. A vertex is called a **leaf** if it has no children. Vertices that have children are called **internal vertices**.

A rooted tree is called an m -ary tree if every internal vertex has no more than m children.

The tree is called a full m -ary tree if every internal vertex has exactly m children.

An m -ary tree with $m = 2$ is called a binary tree.

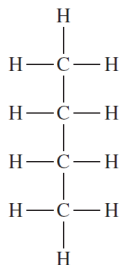


T_1 is full binary tree
 T_2 is full 3-ary tree
 T_3 is full 5-ary tree
 T_4 is 3-ary tree

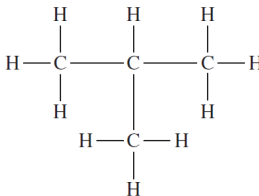
Saturated Hydrocarbons

Trees can be used to represent molecules, where atoms are represented by vertices and bonds between them by edges.

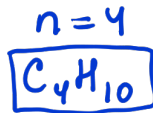
The English mathematician Arthur Cayley discovered trees in 1857 when he was trying to enumerate the isomers of compounds of the form C_nH_{2n+2} , which are called saturated hydrocarbons.



Butane

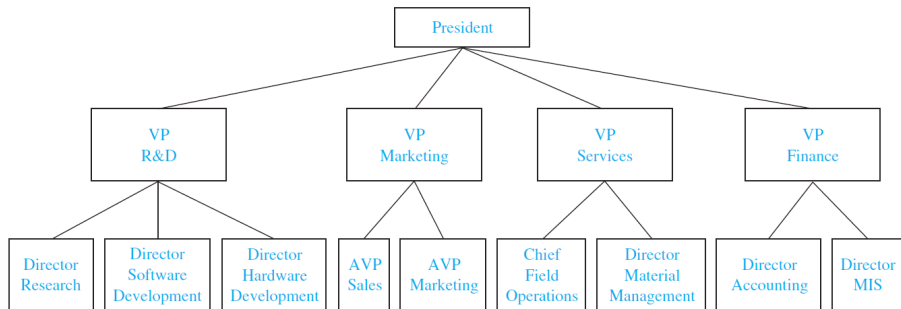


Isobutane

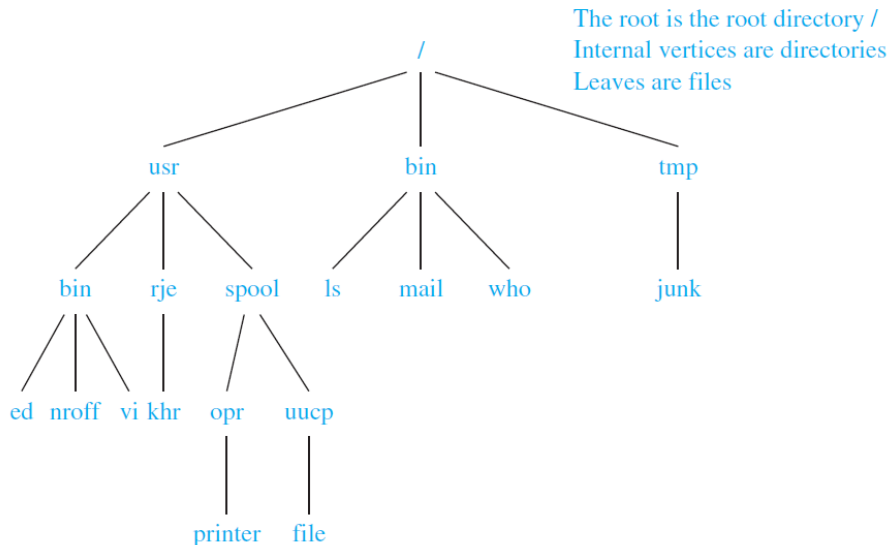


The non-isomorphic trees with n vertices of degree 4 and $2n + 2$ of degree 1 represent the different isomers of C_nH_{2n+2} . For instance, when $n = 4$, there are exactly two non-isomorphic trees of this type.

Representing Organizations

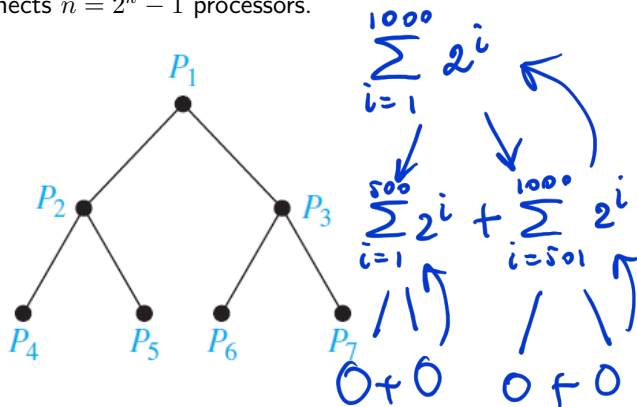


Computer File Systems



Parallel Processors

A **tree-connected network** is an important way to interconnect processors. The graph representing such a network is a binary tree. Such a network interconnects $n = 2^k - 1$ processors.



Properties of trees

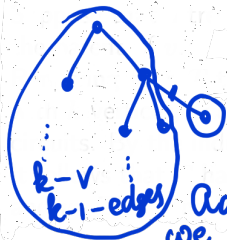
Theorem

"A tree with n vertices has $n - 1$ edges." $= P(n)$, $n \geq 2$

Proof: B.S.: 2 vertex, 1 edge $\Rightarrow P(2) = 1$

I.S. Assume that $P(k) = 1$

Look at tree with $k+1$ vertices
we can find at least 1 leaf
remove it together with the edge
what is left? a tree with k vertices, by the assump.
it has $k-1$ edges

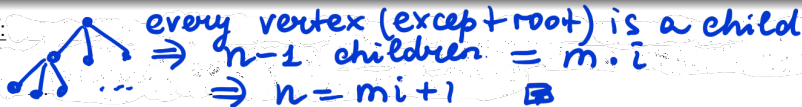


Add that edge and the vertex back.
we get a tree with $k+1$ vertex and
 k edges $\Rightarrow P(k+1) = 1 \Rightarrow$ By PMI $\Rightarrow \forall n P(n) = 1$

Theorem

A full m -ary tree with i internal vertices contains $n = mi + 1$ vertices.

Proof:



Theorem

A full m -ary tree with m, n, i, l if we know any two we can find all others.

- (i) n vertices has $i = (n - 1)/m$ internal vertices and $l = [(m - 1)n + 1]/m$ leaves,
- (ii) i internal vertices has $n = mi + 1$ vertices and $l = (m - 1)i + 1$ leaves,
- (iii) l leaves has $n = (ml - 1)/(m - 1)$ vertices and $i = (l - 1)/(m - 1)$ internal vertices.

Proof:

(i) By thm above $\begin{cases} n = mi + 1 \\ n = i + l \end{cases} \Rightarrow i = \frac{n-1}{m}$

Also we know

$$n = i + l$$

$$l = n - i = n - \frac{n-1}{m} = \frac{mn - n + 1}{m}$$

For exers. show (ii) and (iii)

Example

Suppose that someone starts a chain letter. Each person who receives the letter is asked to send it on to four other people. Some people do this, but others do not send any letters. How many people have seen the letter, including the first person, if no one receives more than one letter and if the chain letter ends after there have been 100 people who read it but did not send it out? How many people sent out the letter?

Answers: Full 4-ary tree, $m=4$, $l=100$

$$\begin{aligned} \text{have seen} &= n \\ \text{sent out} &= i \end{aligned} \quad \begin{cases} n = mi + 1 \\ n = i + l \end{cases} \Rightarrow \begin{aligned} n &= 4i + 1 \\ n &= i + 100 \end{aligned}$$
$$\Rightarrow 4i + 1 = i + 100 \Rightarrow 3i = 99 \Rightarrow i = 33$$
$$n = 133$$

Answer Q1

Homework:

- Solve odd exercises # 1-37 at p. 755.

Tree traversal

(Paragraph 11.3)

Traversal is a process to visit all the nodes of a tree and may use their values. There are three ways which we use to traverse a tree:

- Pre-order Traversal
- In-order Traversal
- Post-order Traversal

Pre-order Traversal:

- Step 1- Visit root node.
- Step 2- Recursively traverse left subtree.
- Step 3- Recursively traverse right subtree.

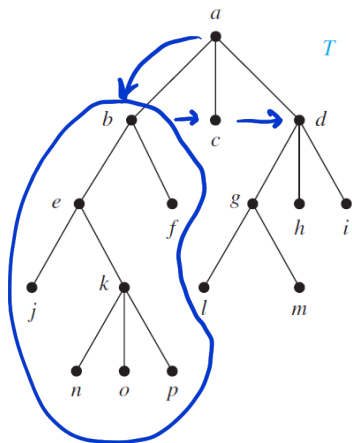
In-order Traversal:

- Step 1- Recursively traverse left subtree.
- Step 2- Visit root node.
- Step 3- Recursively traverse right subtree.

Post-order Traversal:

- Step 1- Recursively traverse left subtree.
- Step 2- Recursively traverse right subtree.
- Step 3- Visit root node.

Pre-order Traversal



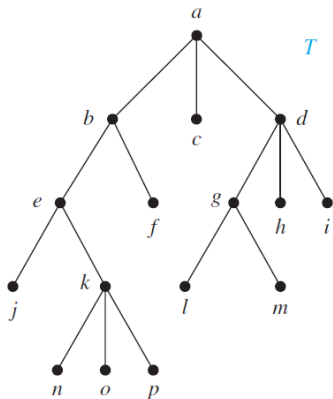
a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i

Algorithm

```
procedure preorder( $T$  : rooted tree)
 $r :=$  root of  $T$ 
list  $r$ 
for each child  $c$  of  $r$  from left to right
 $T(c) :=$  subtree with  $c$  as its root
preorder( $T(c)$ )
```

In-order Traversal

j, e, n, k, o, p, b, f, a, c, l,
g, m, d, h, i



Algorithm

procedure inorder(T : rooted tree)

$r := \text{root of } T$

if r is a leaf then list r

else $l := \text{first child of } r \text{ from left to right}$

$T(l) := \text{subtree with } l \text{ as its root}$
 inorder($T(l)$)

 list r

for each child c of r except for l
 from left to right

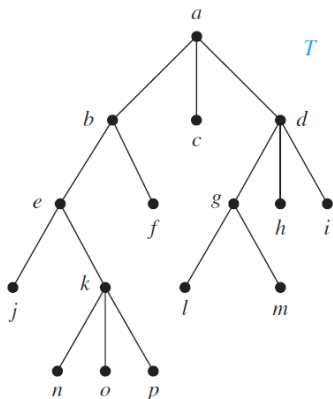
$T(c) := \text{subtree with } c \text{ as its}$

 root

 inorder($T(c)$)

Post-order Traversal

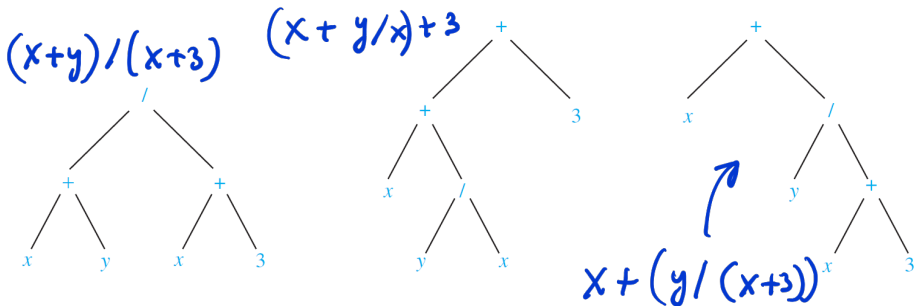
j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a



Algorithm

```
procedure postorder( $T$  : rooted tree)
 $r :=$  root of  $T$ 
for each child  $c$  of  $r$  from left to right
     $T(c) :=$  subtree with  $c$  as its root
    postorder( $T(c)$ )
list  $r$ 
```

Infix form can be ambiguous



In-order traversal (infix form) gives:

It is necessary to include parentheses. $x+y/x+3$

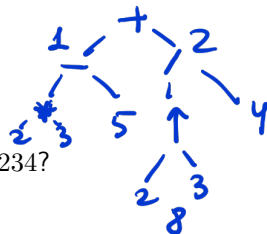
The prefix form (Polish notation) is

$/+xy+x3, ++x/yx3,$
 $+x/y+x3$

The postfix form (reverse Polish notation) is

Both prefix and postfix forms are unambiguous and parentheses are not needed.

Evaluating prefix and postfix expressions



What is the value of the prefix expression $+ - * 2 3 5 / \uparrow 2 3 4$?

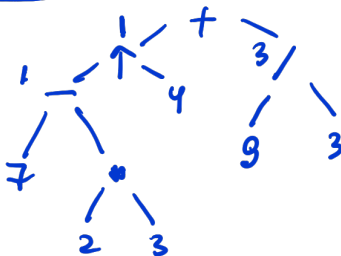
$\uparrow$ is taking a power

Answer: 3

What is the value of the postfix expression $7 2 3 * - 4 \uparrow 9 3 / +$?

Answer: 4

Answer Q2



Homework:

- Solve odd exercises starting at p. 783.